COMPTE RENDU DE TP

Arduino IDE, carte ESP32 et TTGO LoRa32

Installation, configuration et premiers programmes

Réalisé par	**Raja Lassioued** Hibatallah Lafi Jawaher Jaballah Islam Hoch
Classe	LISI 3, groupe C
Matériel	Carte TTGO LoRa32 (ESP32 + LoRa + OLED)
Logiciel	Arduino IDE 2.3.10
Date	23 octobre 2026

Sommaire

1. Introduction et objectifs

2. Présentation du logiciel Arduino IDE

3. Installation de l’Arduino IDE (Windows)

4. Installation de la carte ESP32 dans l’Arduino IDE

5. Installation des bibliothèques

6. Présentation de la carte TTGO LoRa32

7. Programmes de test

8. Conclusion

1. Introduction et objectifs

Ce compte rendu présente la préparation de l'environnement Arduino IDE pour programmer une carte ESP32, en particulier la carte **TTGO LoRa32**, depuis l'installation du logiciel jusqu'aux premiers programmes téléversés et testés.

Objectifs du TP :

- Installer et prendre en main l'Arduino IDE.
- Ajouter le support des cartes ESP32 (gestionnaire de cartes).
- Installer les bibliothèques nécessaires (écran OLED, LoRa).
- Présenter la carte TTGO LoRa32 : composants, caractéristiques, brochage.
- Vérifier l'ensemble avec deux programmes de test : affichage des caractéristiques de la puce et clignotement d'une LED.

2. Présentation du logiciel Arduino IDE

L'Arduino IDE est un environnement de développement gratuit et open source. Il permet d'écrire, compiler et téléverser du code (langage C/C++ simplifié) vers une carte à microcontrôleur, sous Windows, macOS et Linux. Dans ce TP, la version utilisée est l'**IDE 2.3.10**.

Un programme Arduino s'appelle un **croquis** (*sketch*, fichier .ino). Il contient toujours deux fonctions :

- `setup()` : exécutée une seule fois au démarrage (configuration des broches, initialisation).
- `loop()` : exécutée en boucle, sans fin.

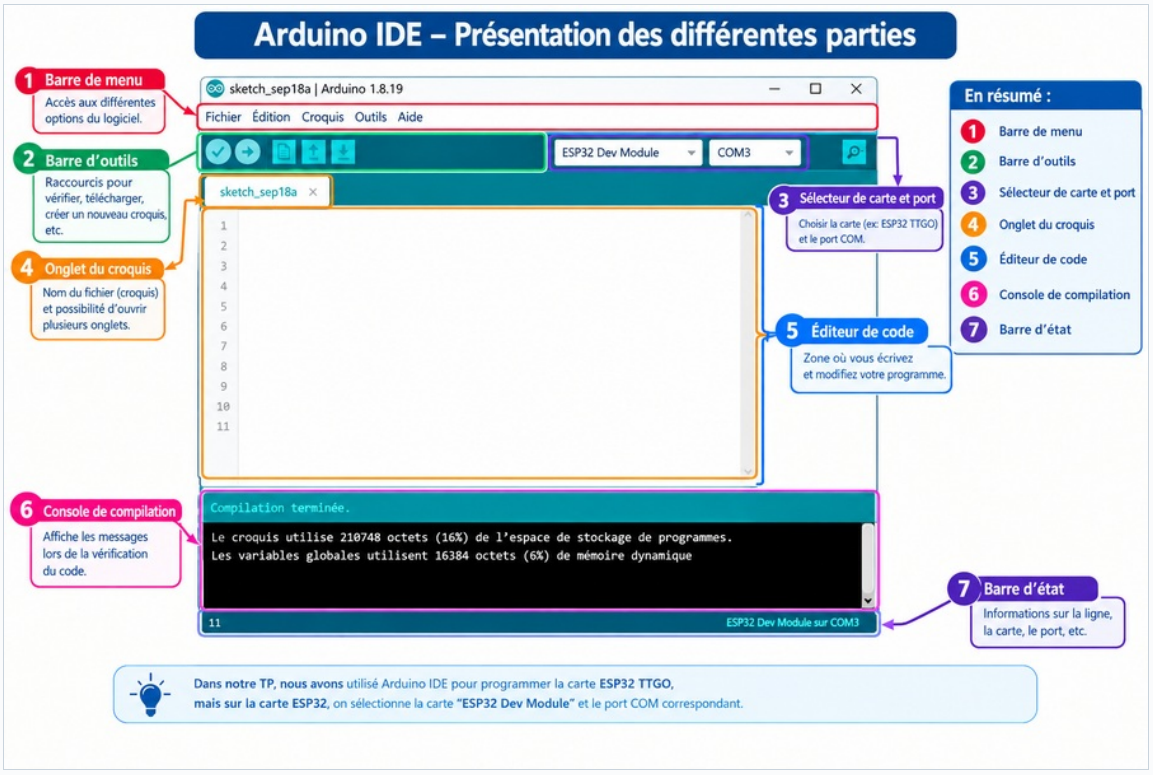


Figure 1. Présentation des différentes parties de l'Arduino IDE (figure du support de TP).

Élément	Rôle
Barre de menu	Accès aux options du logiciel (Fichier, Édition, Croquis, Outils, Aide)
Barre d’outils	Raccourcis : vérifier, téléverser, nouveau croquis
Sélecteur de carte et de port	Choix du modèle de carte et du port COM
Onglet du croquis	Nom du fichier ouvert
Éditeur de code	Zone d’écriture du programme
Console de compilation	Messages de compilation et de téléversement
Barre d’état	Ligne courante, carte et port sélectionnés

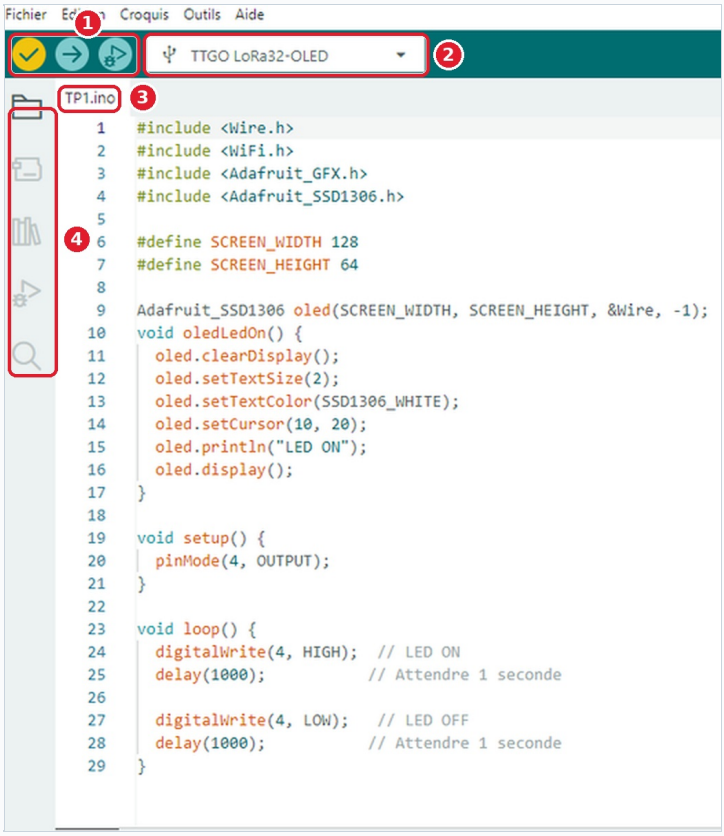


Figure 2. Interface de l’Arduino IDE 2.3.10 (croquis TP1.ino).
(1) barre d’outils : vérifier, téléverser, débogueur ; **(2)** sélecteur de carte et de port (ici « TTGO LoRa32-OLED ») ; **(3)** onglet du croquis ; **(4)** barre latérale : dossier, gestionnaire de cartes, gestionnaire de bibliothèques, débogage, recherche.

3. Installation de l’Arduino IDE (Windows)

1. Ouvrir la page de téléchargement d’Arduino (support.arduino.cc) et choisir **Download the latest release** dans la rubrique **Windows**.
2. Double-cliquer sur le fichier exécutable (.exe) téléchargé.
3. Suivre les instructions de l’assistant d’installation.
4. À la fin de l’installation, laisser cochée l’option **Run Arduino IDE** pour lancer l’application (ou la lancer plus tard depuis le menu Démarrer).

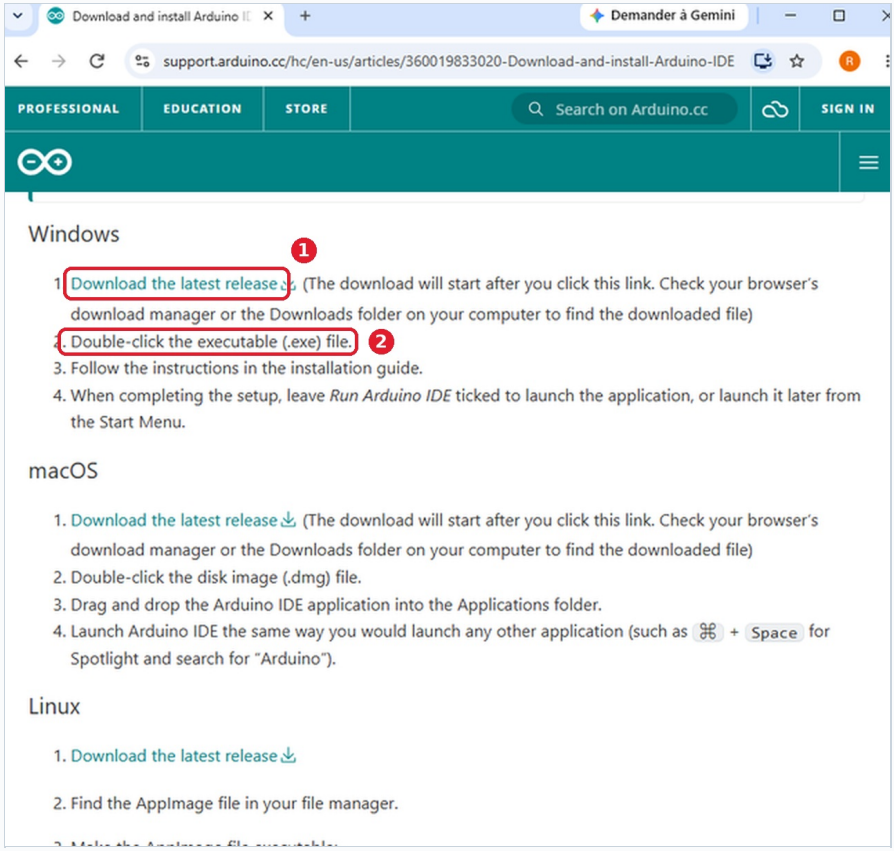


Figure 3. Page de téléchargement de l'Arduino IDE.
(1) lien de téléchargement pour Windows ; **(2)** double-clic sur le fichier .exe pour lancer l'installation.

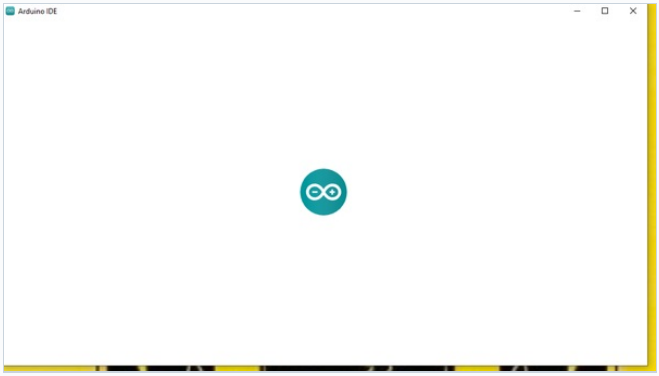


Figure 4. Premier lancement de l'Arduino IDE.

> **Pilote USB de la carte.** La TTGO LoRa32 communique avec le PC par une puce USB-série. Si le port COM n'apparaît pas à la connexion de la carte, installer le pilote correspondant (CP210x de Silicon Labs ou CH340/CH9102 selon la version), puis vérifier dans le *Gestionnaire de périphériques* > *Ports (COM et LPT)*.

4. Installation de la carte ESP32 dans l'Arduino IDE

Par défaut, l'IDE ne connaît pas l'ESP32. Il faut ajouter le support d'Espressif (connexion internet nécessaire, le téléchargement dépasse 200 Mo).

Étape A : ajouter l'URL du gestionnaire de cartes

1. Ouvrir le menu **Fichier > Préférences** (raccourci Ctrl + virgule).

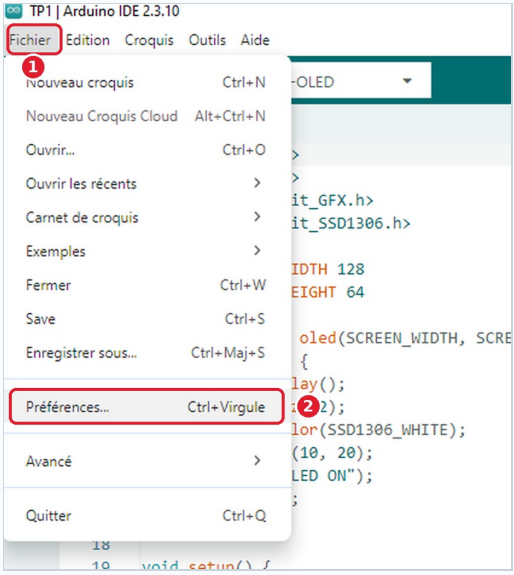


Figure 5. Ouverture des préférences.
(1) menu Fichier ; **(2)** entrée « Préférences… ».

2. Dans le champ **URL de gestionnaire de cartes supplémentaires**, coller :

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

3. Cliquer sur **OK**.

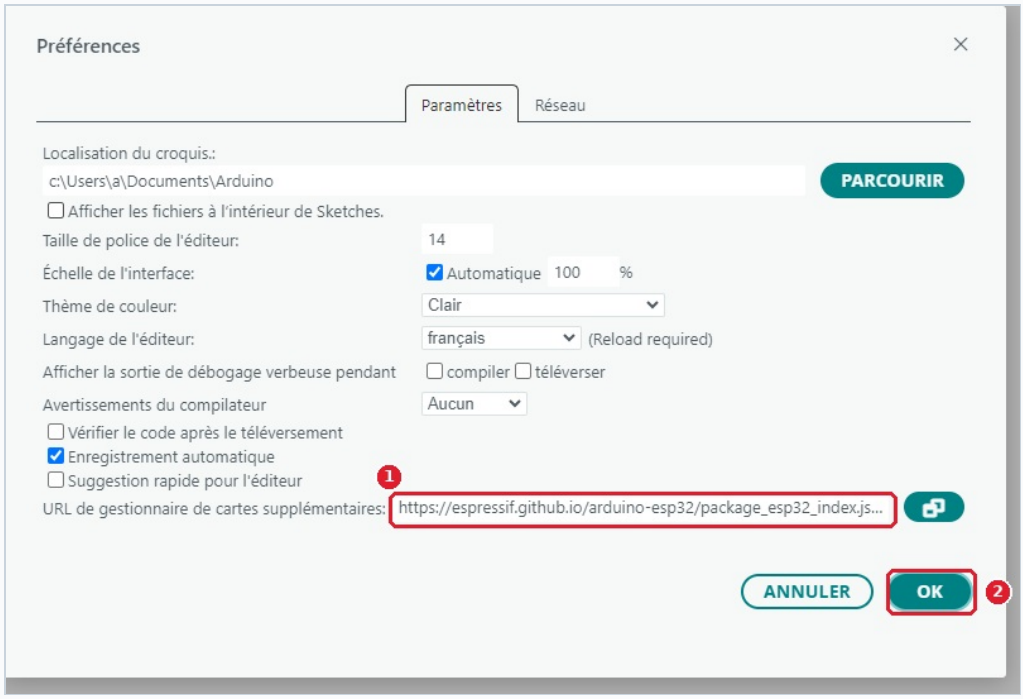


Figure 6. Fenêtre Préférences.
(1) champ « URL de gestionnaire de cartes supplémentaires » contenant l'adresse du paquet ESP32 ; **(2)** bouton OK pour valider.

Étape B : installer le paquet esp32

1. Ouvrir le **Gestionnaire de cartes** (icône de carte dans la barre de gauche, ou *Outils* > *Carte* > *Gestionnaire de cartes*).
2. Taper `esp32` dans la zone de recherche.
3. Choisir le paquet **esp32 par Espressif Systems** et cliquer sur **Installer**. Attendre la fin du téléchargement (plusieurs minutes).
4. Quand le paquet est marqué « installé », les cartes ESP32 sont disponibles.

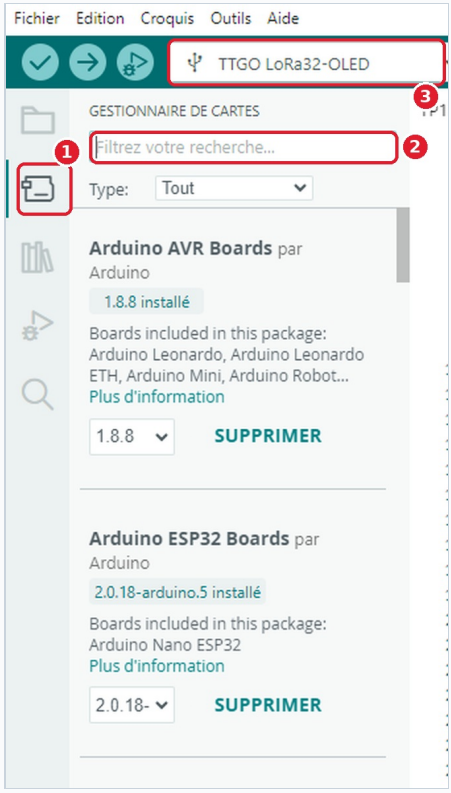


Figure 7. Gestionnaire de cartes.
(1) icône du gestionnaire de cartes ; **(2)** zone de recherche (taper `esp32`) ; **(3)** carte sélectionnée en haut : TTGO LoRa32-OLED.

Étape C : sélectionner la carte et le port

1. Brancher la TTGO LoRa32 au PC avec un câble USB **de données** (certains câbles ne servent qu'à charger).
2. Choisir dans le sélecteur de carte (barre d'outils) : **TTGO LoRa32-OLED**, comme sur la figure 2.
3. Choisir le **port** COM apparu (par exemple COM5).
4. Laisser les autres options par défaut (si le téléversement échoue, réduire la vitesse de téléversement à 115200).

5. Installation des bibliothèques

Une bibliothèque est un ensemble de fonctions prêtes à l'emploi. Pour la TTGO LoRa32, les bibliothèques suivantes sont utilisées :

Bibliothèque	Auteur	Utilité
Adafruit SSD1306	Adafruit	Pilote de l'écran OLED 0,96" (utilisée dans les programmes du TP)
Adafruit GFX Library	Adafruit	Texte et graphismes (requise par SSD1306)
LoRa	Sandeep Mistry	Émission et réception LoRa (pour les applications radio)

Méthode (Gestionnaire de bibliothèques)

1. Ouvrir le **Gestionnaire de bibliothèques** (icône de livres dans la barre de gauche, ou *Croquis > Inclure une bibliothèque > Gérer les bibliothèques*).
2. Rechercher `Adafruit SSD1306` et cliquer sur **Installer**. Si l'IDE propose d'installer aussi les dépendances (Adafruit GFX, Adafruit BusIO), choisir **Tout installer**.
3. Pour la communication radio, rechercher `LoRa` et installer « LoRa » de Sandeep Mistry.
4. Vérifier dans *Croquis > Inclure une bibliothèque* que les bibliothèques figurent dans la liste.

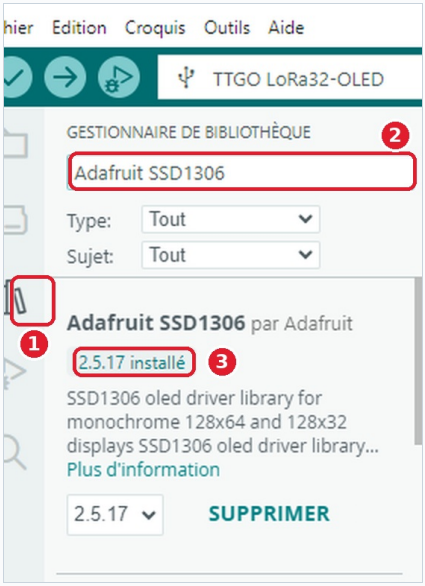


Figure 8. Gestionnaire de bibliothèques.
(1) icône du gestionnaire de bibliothèques ; **(2)** zone de recherche ; **(3)** Adafruit SSD1306 en version 2.5.17, marquée « installé ».

Il est aussi possible d'installer une bibliothèque téléchargée manuellement avec *Croquis > Inclure une bibliothèque > Ajouter la bibliothèque .ZIP*.

6. Présentation de la carte TTGO LoRa32

La TTGO LoRa32 (fabricant LilyGO) est une carte de développement qui réunit sur un seul circuit :

- un microcontrôleur **ESP32** (Wi-Fi et Bluetooth) ;
- un module radio **LoRa** (SX1276/SX1278) pour les communications longue portée à faible consommation ;
- un écran **OLED** 0,96" ;
- un connecteur de batterie Li-Po avec circuit de charge.

Elle convient aux projets IoT : capteurs distants, communication entre deux cartes, passerelles simples.

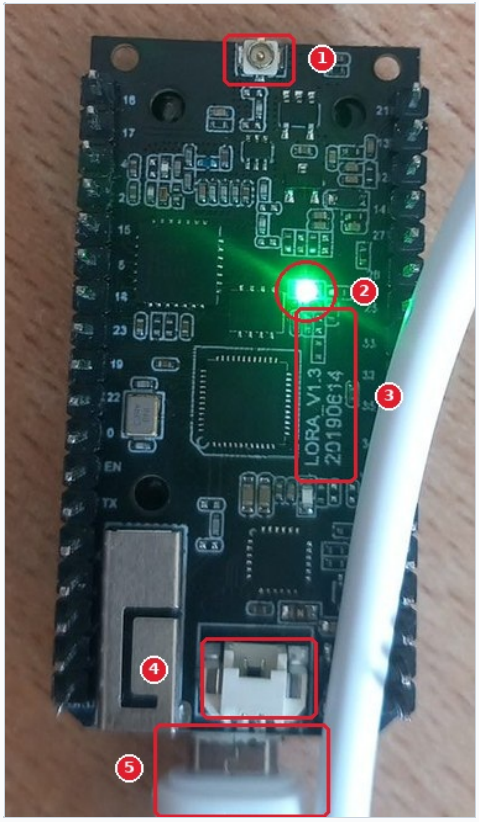


Figure 9. Carte utilisée en TP (face arrière).
(1) connecteur d'antenne U.FL ; **(2)** LED verte allumée (carte alimentée) ; **(3)** inscription « LORA_V1.3 20190614 » ; **(4)** connecteur de batterie ; **(5)** prise micro-USB.

Caractéristiques principales

Caractéristique	Valeur
Microcontrôleur	ESP32-D0WDQ6-V3, double cœur, jusqu'à 240 MHz (modèle et nombre de cœurs lus sur l'OLED)
Mémoire flash	4 Mo (valeur lue sur l'OLED)
Mémoire vive	520 Ko de SRAM (224 Ko libres au démarrage du programme de test)
Connectivité	Wi-Fi 802.11 b/g/n, Bluetooth 4.2 classique et BLE
Radio LoRa	SX1276 ou SX1278, bande 433, 868 ou 915 MHz selon le modèle
Écran	OLED 0,96" 128×64, contrôleur SSD1306, bus I2C (adresse 0x3C)
Alimentation	USB 5 V (micro-USB) ou batterie Li-Po 3,7 V

Broches utilisées dans ce TP

Fonction	Broche ESP32
OLED SDA	GPIO 21
OLED SCL	GPIO 22
LED externe (montage sur breadboard)	GPIO 4

Pour le module LoRa, les broches SPI courantes sont SCK = GPIO 5, MISO = GPIO 19, MOSI = GPIO 27, CS = GPIO 18 et DIO0 = GPIO 26. La broche RST du module LoRa varie selon la version de la carte : elle doit être vérifiée sur le schéma du constructeur avant tout programme LoRa.

Précaution. Ne jamais faire émettre le module LoRa sans l’antenne branchée : l’émission sans antenne peut abîmer le module.

7. Programmes de test

7.1 Programme 1 : caractéristiques de la puce sur l’OLED et le moniteur série

Ce programme affiche sur l’écran OLED le modèle de la puce, le nombre de cœurs, la mémoire flash, la mémoire libre et l’adresse MAC. L’adresse MAC est aussi envoyée sur le moniteur série (115200 bauds).

```
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 oled(128, 64, &Wire, -1);  // OLED 128x64, pas de reset

void setup() {
  Serial.begin(115200);
  Wire.begin(21, 22);                       // SDA, SCL de l'OLED
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  oled.setTextColor(SSD1306_WHITE);
  WiFi.mode(WIFI_STA);
  delay(500);

  oled.clearDisplay();
  oled.setTextSize(1);
  oled.setCursor(0, 0);
  oled.println("== Infos ESP32 ==");
  oled.printf("Modele: %s\n", ESP.getChipModel());
  oled.printf("Coeurs: %d  %d MHz\n", ESP.getChipCores(), ESP.getCpuFreqMHz());
  oled.printf("Flash : %d Mo\n", ESP.getFlashChipSize() / (1024 * 1024));
  oled.printf("RAM   : %d o\n", ESP.getFreeHeap());
  oled.println("MAC:");
  oled.println(WiFi.macAddress());
  oled.display();
  Serial.println("MAC : " + WiFi.macAddress());
}

void loop() {
}
```

Explication du code (Programme 1)

Instruction	Explication
`#include <WiFi.h>`	Bibliothèque Wi-Fi de l'ESP32, utilisée ici pour lire l'adresse MAC.
`#include <Wire.h>`	Bibliothèque du bus I2C, utilisé pour communiquer avec l'écran OLED.
`#include <Adafruit_GFX.h>` et `<Adafruit_SSD1306.h>`	Bibliothèques graphiques et pilote de l'écran OLED SSD1306.
`Adafruit_SSD1306 oled(128, 64, &Wire, -1);`	Crée l'objet `oled` : écran de 128×64 pixels, relié au bus I2C `Wire`. Le `-1` signifie qu'aucune broche de reset n'est utilisée.
`Serial.begin(115200);`	Ouvre la liaison série avec le PC à 115200 bauds (affichage dans le moniteur série).
`Wire.begin(21, 22);`	Démarre le bus I2C avec SDA = GPIO 21 et SCL = GPIO 22.
`oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);`	Initialise l'écran : alimentation interne de l'écran activée, adresse I2C 0x3C.
`oled.setTextColor(SSD1306_WHITE);`	Texte en blanc (couleur d'allumage des pixels).
`WiFi.mode(WIFI_STA);`	Active le module Wi-Fi en mode « station », ce qui permet de lire l'adresse MAC.
`delay(500);`	Pause de 500 ms pour laisser le temps à l'écran et au Wi-Fi de démarrer.
`oled.clearDisplay();`	Efface la mémoire d'affichage de l'écran.
`oled.setTextSize(1);`	Taille de texte normale (petite).
`oled.setCursor(0, 0);`	Place le curseur en haut à gauche.
`oled.println(...)` et `oled.printf(...)`	Écrivent le texte dans la mémoire d'affichage. `printf` permet d'insérer des valeurs : `%s` pour un texte, `%d` pour un entier.
`ESP.getChipModel()`	Nom du modèle de la puce (ici ESP32-D0WDQ6-V3).
`ESP.getChipCores()`	Nombre de cœurs du processeur (2).
`ESP.getCpuFreqMHz()`	Fréquence du processeur en MHz.
`ESP.getFlashChipSize() / (1024 * 1024)`	Taille de la mémoire flash, convertie d'octets en Mo.
`ESP.getFreeHeap()`	Mémoire vive libre, en octets.
`WiFi.macAddress()`	Adresse MAC de la carte (identifiant réseau unique, 6 octets).
`oled.display();`	Envoie le contenu de la mémoire d'affichage vers l'écran : sans cette ligne, rien ne s'affiche.
`Serial.println("MAC : " + WiFi.macAddress());`	Envoie l'adresse MAC vers le moniteur série.
`void loop() { }`	Vide : tout le travail est fait une seule fois dans `setup()`.

Principe de fonctionnement. Au démarrage, `setup()` initialise la liaison série, le bus I2C, l'écran et le Wi-Fi. Le programme écrit ensuite les informations de la puce dans la mémoire d'affichage, puis les envoie à l'écran avec `oled.display()`. Comme `loop()` est vide, l'affichage reste fixe jusqu'à un reset de la carte.

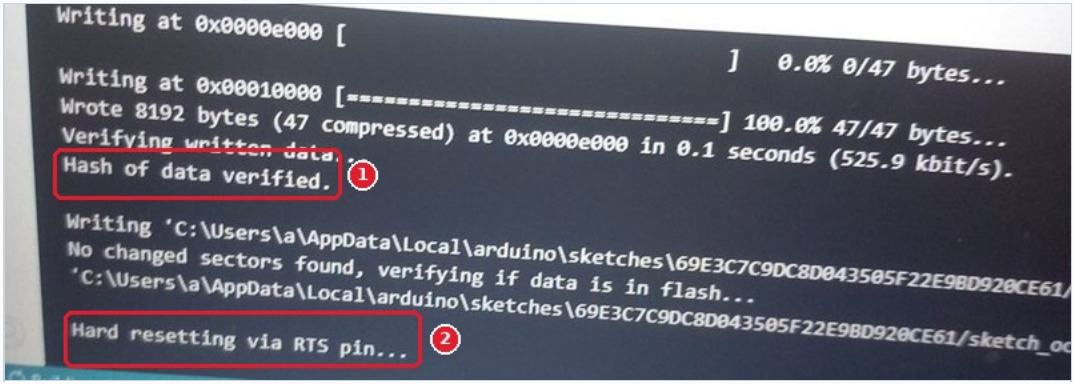


Figure 10. Fin du téléversement dans la console de l'IDE.
(1) « Hash of data verified » : les données écrites dans la mémoire flash sont correctes ; **(2)** « Hard resetting via RTS pin » : la carte redémarre et exécute le programme.



Figure 11. Résultat affiché sur l'écran OLED.
(1) modèle de la puce ; **(2)** nombre de cœurs ; **(3)** mémoire flash ; **(4)** mémoire vive libre ; **(5)** adresse MAC.

Information	Valeur affichée
Modèle	ESP32-D0WDQ6-V3
Cœurs	2
Flash	4 Mo
RAM libre	224 Ko
Adresse MAC	A0:DD:6C:67:C2:88

Interprétation des résultats.

- **ESP32-D0WDQ6-V3** : puce ESP32 double cœur, révision 3.
- **2 cœurs** : le processeur peut exécuter deux tâches en parallèle.
- **4 Mo de flash** : mémoire où est stocké le programme téléversé.
- **224 Ko de RAM libre** : sur les 520 Ko de SRAM de la puce, le reste est utilisé par le système, la pile Wi-Fi et le programme.
- **Adresse MAC** : identifiant unique de la carte sur un réseau, formé de 6 octets en hexadécimal.

7.2 Programme 2 : clignotement d'une LED avec affichage OLED

Une LED, associée à une résistance de protection, est branchée sur la broche GPIO 4 de la carte (montage sur breadboard). Le programme allume la LED pendant 1 seconde en affichant « LED ON » sur l'écran OLED, puis l'éteint

pendant 1 seconde en affichant « LED OFF », en boucle.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define LED_PIN 4

Adafruit_SSD1306 oled(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void oledLedOn() {
  oled.clearDisplay();
  oled.setTextSize(2);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(10, 20);
  oled.println("LED ON");
  oled.display();
}

void oledLedOff() {
  oled.clearDisplay();
  oled.setTextSize(2);
  oled.setTextColor(SSD1306_WHITE);
  oled.setCursor(10, 20);
  oled.println("LED OFF");
  oled.display();
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  Wire.begin(21, 22);                        // SDA, SCL de l'OLED
  oled.begin(SSD1306_SWITCHCAPVCC, 0x3C);
}

void loop() {
  digitalWrite(LED_PIN, HIGH);  // LED ON
  oledLedOn();
  delay(1000);                  // Attendre 1 seconde

  digitalWrite(LED_PIN, LOW);   // LED OFF
  oledLedOff();
  delay(1000);                  // Attendre 1 seconde
}
```

Explication du code (Programme 2)

Instruction	Explication
`#define SCREEN_WIDTH 128` et `SCREEN_HEIGHT 64`	Constantes donnant la taille de l'écran OLED en pixels.
`#define LED_PIN 4`	Donne un nom à la broche de la LED (GPIO 4), pour rendre le code plus lisible.
`Adafruit_SSD1306 oled(...)`	Crée l'objet écran, comme dans le programme 1.
`void oledLedOn()`	Fonction qui efface l'écran, puis affiche « LED ON » en grand texte (taille 2) à la position (10, 20) et envoie le tout à l'écran avec `oled.display()`.
`void oledLedOff()`	Même principe, pour afficher « LED OFF ».
`pinMode(LED_PIN, OUTPUT);`	Configure la broche GPIO 4 en sortie, pour commander la LED.
`Wire.begin(21, 22);` et `oled.begin(...)`	Démarrent le bus I2C (SDA = 21, SCL = 22) puis initialisent l'écran à l'adresse 0x3C. Sans ces lignes, l'écran n'affiche rien.
`digitalWrite(LED_PIN, HIGH);`	Met la broche à l'état haut (environ 3,3 V) : le courant traverse la résistance et la LED, qui s'allume.
`oledLedOn();`	Affiche « LED ON » en même temps que la LED s'allume.
`delay(1000);`	Attend 1000 ms (1 seconde).
`digitalWrite(LED_PIN, LOW);`	Met la broche à l'état bas (0 V) : la LED s'éteint.
`oledLedOff();`	Affiche « LED OFF » en même temps que la LED s'éteint.
`loop()`	Se répète indéfiniment : 1 s allumée, 1 s éteinte (période de 2 s, fréquence de 0,5 Hz).

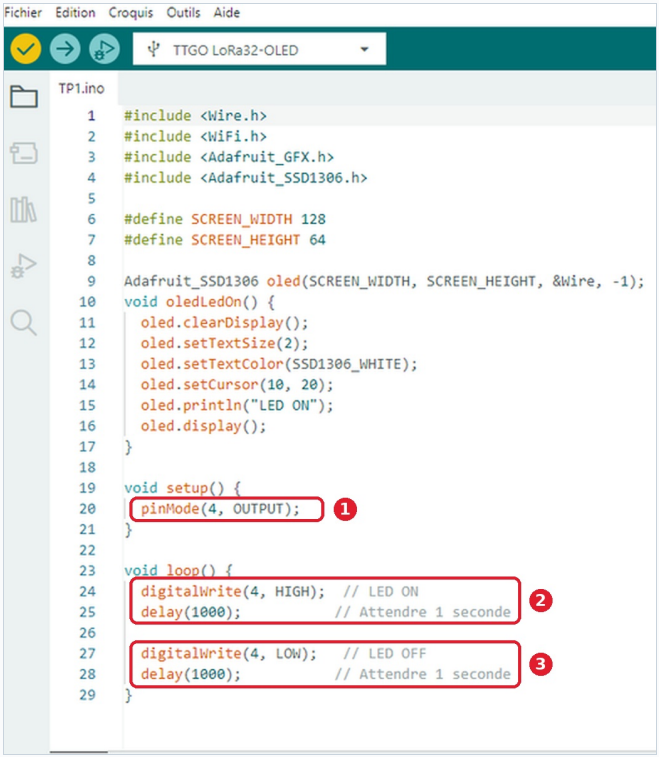


Figure 12. Croquis de départ TP1.ino dans l'IDE : commande de la broche 4 et fonction `oledLedOn()`. La version complète ci-dessus y ajoute l'initialisation de l'écran et l'affichage « LED ON » / « LED OFF » dans `loop()`.

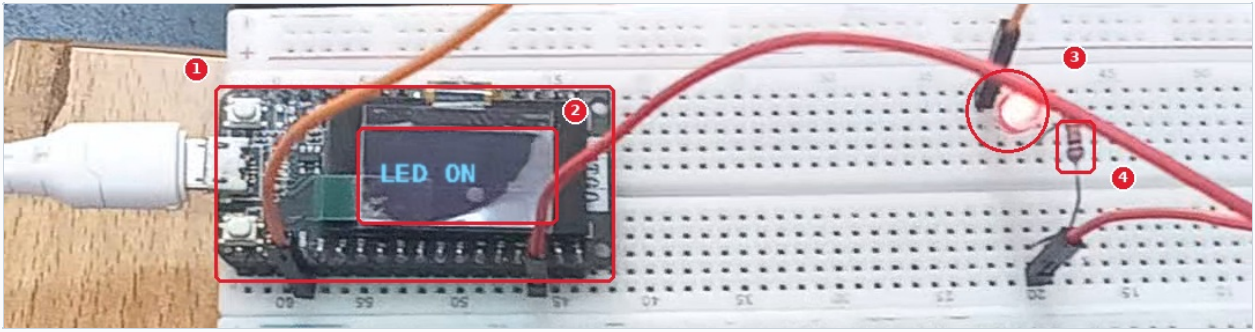


Figure 13. Montage, LED allumée : l'OLED affiche « LED ON ».
(1) carte TTGO LoRa32 ; **(2)** écran OLED ; **(3)** LED rouge ; **(4)** résistance de protection. *Image issue de la vidéo du montage ; le texte de l'écran OLED est ajouté numériquement pour illustrer le programme.*

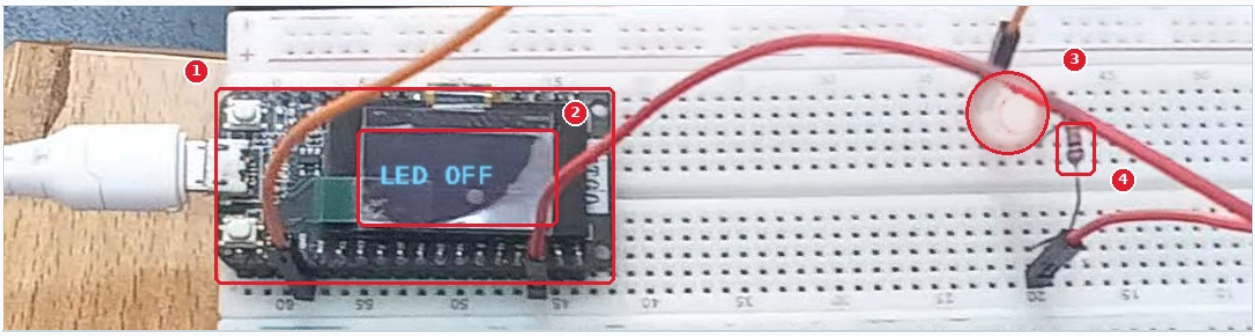


Figure 14. Montage, LED éteinte : l'OLED affiche « LED OFF ».
(1) carte TTGO LoRa32 ; **(2)** écran OLED ; **(3)** LED éteinte ; **(4)** résistance de protection. *Image issue de la vidéo du montage ; le texte de l'écran OLED est ajouté numériquement pour illustrer le programme.*

État	Broche GPIO 4	LED	Affichage OLED
Allumé (1 s)	HIGH (3,3 V)	allumée	LED ON
Éteint (1 s)	LOW (0 V)	éteinte	LED OFF

7.3 Problèmes fréquents

Problème	Cause probable	Solution
Port COM absent	Câble sans données ou pilote manquant	Changer de câble, installer le pilote USB-série
« Failed to connect to ESP32 »	La carte n'entre pas en mode téléversement	Maintenir le bouton BOOT pendant l'envoi, puis relâcher
« No such file » à la compilation	Bibliothèque manquante	Installer la bibliothèque via le Gestionnaire de bibliothèques
Caractères illisibles dans le moniteur série	Mauvaise vitesse de communication	Régler le moniteur série sur 115200 bauds
OLED reste noir	Mauvaises broches I2C ou adresse	Utiliser SDA = 21, SCL = 22 et l'adresse 0x3C

8. Conclusion

Ce TP a permis de configurer un environnement complet : Arduino IDE installé, support ESP32 ajouté via le gestionnaire de cartes, bibliothèques installées, puis carte TTGO LoRa32 testée avec un programme qui affiche les caractéristiques de la puce sur l'OLED et sur le moniteur série, et avec un programme de clignotement de LED accompagné de l'affichage « LED ON » / « LED OFF » sur l'écran OLED. Cette base permet de passer à des applications plus avancées, comme la liaison entre deux cartes (émetteur et récepteur LoRa) ou l'envoi de mesures de capteurs à distance.